

```
import os
import kagglehub

# Set Kaggle API credentials
os.environ["KAGGLE_USERNAME"] = "bassimanjula"
os.environ["KAGGLE_API_TOKEN"] = "KGAT_243799148244108ea1e8e41f531ac577"

# Download the dataset
store_sales_time_series_forecasting_path = kagglehub.competition_download('store-sales-time-series-forecasting')
print('Data source import complete.')
```

Downloading to /root/.cache/kagglehub/competitions/store-sales-time-series-forecasting.archive...
 100%|██████████| 21.4M/21.4M [00:00<00:00, 62.3MB/s]Extracting files...

Data source import complete.

```
import os
import pandas as pd

# List all files downloaded from the dataset
print("Downloaded files:")
for file_name in os.listdir(store_sales_time_series_forecasting_path):
    print("-", file_name)

# Load the main dataset files
train_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'train.csv'))
stores_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'stores.csv'))

# Display first few rows of train data
train_df.head()
```

Downloaded files:
 - sample_submission.csv
 - holidays_events.csv
 - stores.csv
 - oil.csv
 - transactions.csv
 - test.csv
 - train.csv

	id	date	store_nbr	family	sales	onpromotion	
0	0	2013-01-01	1	AUTOMOTIVE	0.0	0	
1	1	2013-01-01	1	BABY CARE	0.0	0	
2	2	2013-01-01	1	BEAUTY	0.0	0	
3	3	2013-01-01	1	BEVERAGES	0.0	0	
4	4	2013-01-01	1	BOOKS	0.0	0	

```
# Load all additional CSV files
oil_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'oil.csv'))
holidays_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'holidays_events.csv'))
transactions_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'transactions.csv'))

# Merge store metadata into the training set
merged_df = train_df.merge(stores_df, on='store_nbr', how='left')

# Check structure and missing values
print("Dataset Shape:", merged_df.shape)
merged_df.info()
```

Dataset Shape: (3000888, 10)
 <class 'pandas.core.frame.DataFrame'>
 RangeIndex: 3000888 entries, 0 to 3000887
 Data columns (total 10 columns):
 # Column Dtype
 --- ---
 0 id int64
 1 date object
 2 store_nbr int64
 3 family object
 4 sales float64

```
5  onpromotion  int64
6  city         object
7  state        object
8  type         object
9  cluster      int64
dtypes: float64(1), int64(4), object(5)
memory usage: 228.9+ MB
```

```
import matplotlib.pyplot as plt
import seaborn as sns

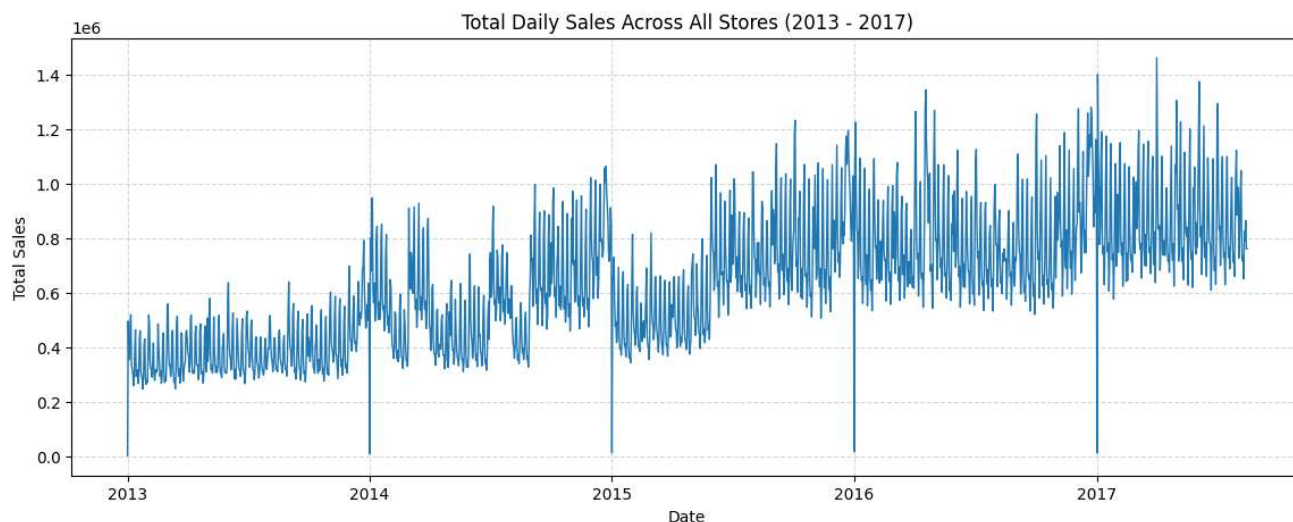
# Convert date column to datetime
merged_df['date'] = pd.to_datetime(merged_df['date'])

# Group sales by date to observe global trend over time
daily_sales = merged_df.groupby('date')['sales'].sum().reset_index()

# Plot Total Daily Sales
plt.figure(figsize=(14, 5))
plt.plot(daily_sales['date'], daily_sales['sales'], color='tab:blue', linewidth=1)
plt.title('Total Daily Sales Across All Stores (2013 - 2017)')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

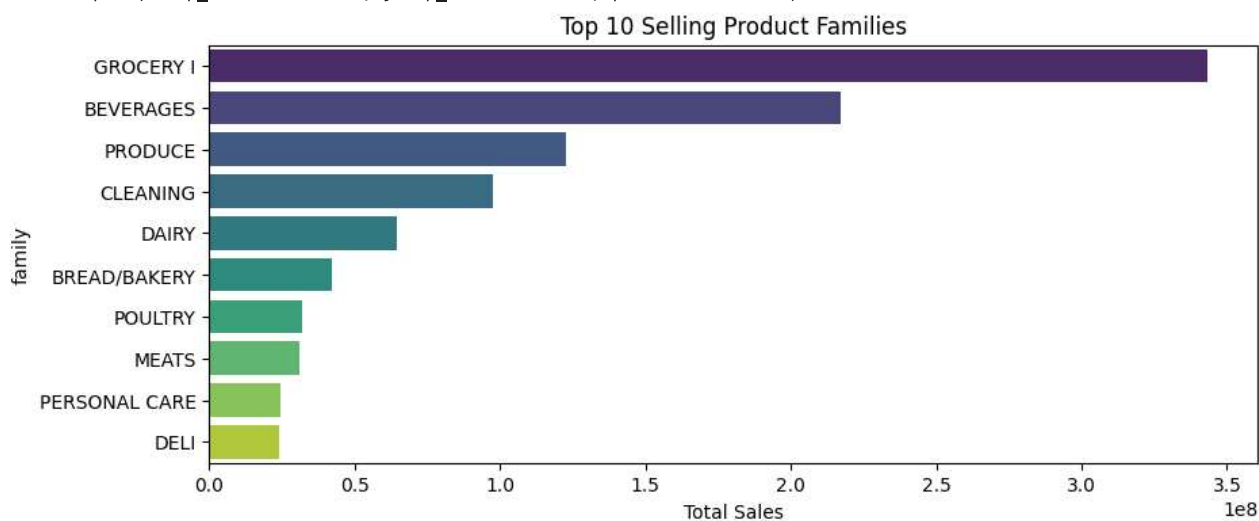
# Top 10 Product Families by Total Sales
top_families = merged_df.groupby('family')['sales'].sum().sort_values(ascending=False).head(10)

plt.figure(figsize=(10, 4))
sns.barplot(x=top_families.values, y=top_families.index, palette='viridis')
plt.title('Top 10 Selling Product Families')
plt.xlabel('Total Sales')
plt.show()
```



/tmp/ipykernel_5104/1197468678.py:23: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and
 sns.barplot(x=top_families.values, y=top_families.index, palette='viridis')



```
import numpy as np

# 1. Date Feature Engineering
merged_df['year'] = merged_df['date'].dt.year
merged_df['month'] = merged_df['date'].dt.month
merged_df['day'] = merged_df['date'].dt.day
merged_df['dayofweek'] = merged_df['date'].dt.dayofweek
merged_df['is_weekend'] = merged_df['dayofweek'].isin([5, 6]).astype(int)

# 2. Handle missing values in Oil prices
oil_df['date'] = pd.to_datetime(oil_df['date'])
# Fill missing dates/prices with forward and backward fill
oil_df['dcoilwtico'] = oil_df['dcoilwtico'].ffill().bfill()

# Merge oil data into main dataset
merged_df = merged_df.merge(oil_df, on='date', how='left')
merged_df['dcoilwtico'] = merged_df['dcoilwtico'].ffill().bfill()

# 3. Create Lag Features for Sales (captures immediate historical sales dynamics)
merged_df = merged_df.sort_values(by=['store_nbr', 'family', 'date'])
merged_df['sales_lag_7'] = merged_df.groupby(['store_nbr', 'family'])['sales'].shift(7)
merged_df['sales_lag_14'] = merged_df.groupby(['store_nbr', 'family'])['sales'].shift(14)

# Display processed feature preview
merged_df[['date', 'store_nbr', 'family', 'sales', 'dcoilwtico', 'sales_lag_7']].head(10)
```

	date	store_nbr	family	sales	dcoilwtico	sales_lag_7	
0	2013-01-01	1	AUTOMOTIVE	0.0	93.14	NaN	
1782	2013-01-02	1	AUTOMOTIVE	2.0	93.14	NaN	
3564	2013-01-03	1	AUTOMOTIVE	3.0	92.97	NaN	
5346	2013-01-04	1	AUTOMOTIVE	3.0	93.12	NaN	
7128	2013-01-05	1	AUTOMOTIVE	5.0	93.12	NaN	
8910	2013-01-06	1	AUTOMOTIVE	2.0	93.12	NaN	
10692	2013-01-07	1	AUTOMOTIVE	0.0	93.20	NaN	
12474	2013-01-08	1	AUTOMOTIVE	2.0	93.21	0.0	
14256	2013-01-09	1	AUTOMOTIVE	2.0	93.08	2.0	
16038	2013-01-10	1	AUTOMOTIVE	2.0	93.81	3.0	

```
import lightgbm as lgb
from sklearn.metrics import root_mean_squared_error
import numpy as np

# 1. Drop initial rows containing NaN values from lag creation
df_model = merged_df.dropna(subset=['sales_lag_7', 'sales_lag_14']).copy()

# 2. Encode categorical variables
df_model['family'] = df_model['family'].astype('category')
df_model['store_nbr'] = df_model['store_nbr'].astype('category')
df_model['city'] = df_model['city'].astype('category')
df_model['state'] = df_model['state'].astype('category')
df_model['type'] = df_model['type'].astype('category')

# 3. Train-Validation Split (Use last 16 days as validation set, matching test period)
max_date = df_model['date'].max()
split_date = max_date - pd.Timedelta(days=15)

train_data = df_model[df_model['date'] < split_date]
val_data = df_model[df_model['date'] >= split_date]

features = ['store_nbr', 'family', 'onpromotion', 'city', 'state', 'type', 'cluster',
            'year', 'month', 'day', 'dayofweek', 'is_weekend', 'dcoilwtico',
            'sales_lag_7', 'sales_lag_14']

X_train, y_train = train_data[features], train_data['sales']
X_val, y_val = val_data[features], val_data['sales']

# Use log1p transformation on sales target for RMSLE optimization
y_train_log = np.log1p(y_train)
y_val_log = np.log1p(y_val)

# 4. Train LightGBM Regressor
model = lgb.LGBMRegressor(
    n_estimators=500,
    learning_rate=0.05,
    num_leaves=31,
    random_state=42
)

model.fit(
    X_train, y_train_log,
    eval_set=[(X_val, y_val_log)],
    callbacks=[lgb.early_stopping(stopping_rounds=50, verbose=100)]
)

# 5. Evaluate Model Validation Performance
val_preds_log = model.predict(X_val)
val_preds_log = np.clip(val_preds_log, 0, None) # Ensure no negative sales predictions

rmsle = root_mean_squared_error(y_val_log, val_preds_log)
print(f"\nValidation RMSLE Score: {rmsle:.4f}")
```

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.296721 seconds.
 You can set `force\_row\_wise=true` to remove the overhead.
 And if memory is not enough, you can set `force\_col\_wise=true`.

```
[LightGBM] [Info] Total Bins 1179
[LightGBM] [Info] Number of data points in the train set: 2947428, number of used features: 15
[LightGBM] [Info] Start training from score 2.927778
Training until validation scores don't improve for 50 rounds
Did not meet early stopping. Best iteration is:
[500]  valid_0's l2: 0.183598
```

Validation RMSLE Score: 0.4285

```
# 1. Load test dataset
test_df = pd.read_csv(os.path.join(store_sales_time_series_forecasting_path, 'test.csv'))
test_df['date'] = pd.to_datetime(test_df['date'])

# 2. Merge store data into test set
test_df = test_df.merge(stores_df, on='store_nbr', how='left')

# 3. Process date features
test_df['year'] = test_df['date'].dt.year
test_df['month'] = test_df['date'].dt.month
test_df['day'] = test_df['date'].dt.day
test_df['dayofweek'] = test_df['date'].dt.dayofweek
test_df['is_weekend'] = test_df['dayofweek'].isin([5, 6]).astype(int)

# 4. Merge oil prices
test_df = test_df.merge(oil_df, on='date', how='left')
test_df['dcoilwtico'] = test_df['dcoilwtico'].ffill().bfill()

# 5. Attach latest available sales lag features from training set
all_df = pd.concat([merged_df, test_df], ignore_index=True)
all_df = all_df.sort_values(by=['store_nbr', 'family', 'date'])

all_df['sales_lag_7'] = all_df.groupby(['store_nbr', 'family'])['sales'].shift(7)
all_df['sales_lag_14'] = all_df.groupby(['store_nbr', 'family'])['sales'].shift(14)

# Filter test set with engineered lag features
test_processed = all_df[all_df['id'].isin(test_df['id'])].copy()

# Ensure categoricals match model requirements
test_processed['family'] = test_processed['family'].astype('category')
test_processed['store_nbr'] = test_processed['store_nbr'].astype('category')
test_processed['city'] = test_processed['city'].astype('category')
test_processed['state'] = test_processed['state'].astype('category')
test_processed['type'] = test_processed['type'].astype('category')

# 6. Predict on Test Set & Convert back from log1p scale
test_preds_log = model.predict(test_processed[features])
test_preds = np.expml(np.clip(test_preds_log, 0, None))

# Create final submission dataframe
submission = pd.DataFrame({
    'id': test_processed['id'].astype(int),
    'sales': test_preds
}).sort_values('id')

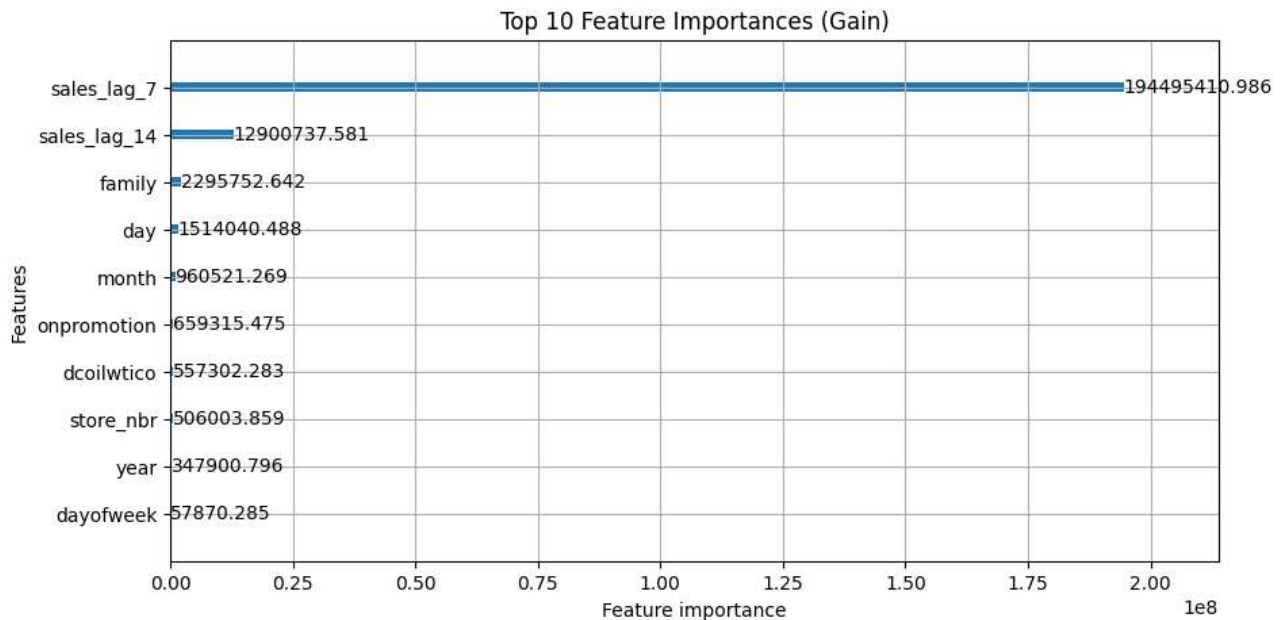
submission.to_csv('submission.csv', index=False)
print("Submission file successfully generated!")
submission.head()
```

Submission file successfully generated!

	id	sales
3000888	3000888	3.712195
3000889	3000889	0.028674
3000890	3000890	3.974982
3000891	3000891	2354.751008
3000892	3000892	0.036505

Next steps: [Generate code with submission](#) [New interactive sheet](#)

```
# Plot Top Feature Importances
lgb.plot_importance(model, max_num_features=10, importance_type='gain', figsize=(10, 5))
plt.title('Top 10 Feature Importances (Gain)')
plt.show()
```



```
# 1. Add Rolling Window Features & Holiday Signals
merged_df = merged_df.sort_values(by=['store_nbr', 'family', 'date'])

# Rolling means capture recent short-term and medium-term sales momentum
merged_df['sales_rolling_mean_7'] = merged_df.groupby(['store_nbr', 'family'])['sales'].transform(lambda x: x.shift(7).rolling(7).mean())
merged_df['sales_rolling_mean_14'] = merged_df.groupby(['store_nbr', 'family'])['sales'].transform(lambda x: x.shift(7).rolling(14).mean())

# Process Holidays
holidays_df['date'] = pd.to_datetime(holidays_df['date'])
national_holidays = holidays_df[holidays_df['transferred'] == False]['date'].unique()
merged_df['is_holiday'] = merged_df['date'].isin(national_holidays).astype(int)

# Drop rows with NaNs from rolling window creation
df_model_v2 = merged_df.dropna(subset=['sales_rolling_mean_14']).copy()

# Categoricals
for col in ['family', 'store_nbr', 'city', 'state', 'type']:
    df_model_v2[col] = df_model_v2[col].astype('category')

# Train/Val Split
split_date = df_model_v2['date'].max() - pd.Timedelta(days=15)
train_v2 = df_model_v2[df_model_v2['date'] < split_date]
val_v2 = df_model_v2[df_model_v2['date'] >= split_date]

features_v2 = ['store_nbr', 'family', 'onpromotion', 'city', 'state', 'type', 'cluster',
               'year', 'month', 'day', 'dayofweek', 'is_weekend', 'dcoilwtico',
               'sales_lag_7', 'sales_lag_14', 'sales_rolling_mean_7', 'sales_rolling_mean_14', 'is_holiday']

X_tr, y_tr_log = train_v2[features_v2], np.log1p(train_v2['sales'])
X_va, y_va_log = val_v2[features_v2], np.log1p(val_v2['sales'])

# 2. Retrain LightGBM Model
model_v2 = lgb.LGBMRegressor(
    n_estimators=1000,
    learning_rate=0.03,
    num_leaves=63,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

model_v2.fit(
    X_tr, y_tr_log,
    eval_set=[(X_va, y_va_log)],
    callbacks=[lgb.early_stopping(stopping_rounds=50, verbose=100)]
)

# 3. New RMSLE
```

```
val_preds_v2 = np.clip(model_v2.predict(X_va), 0, None)
new_rmsle = root_mean_squared_error(y_va_log, val_preds_v2)
print(f"\nImproved Validation RMSLE Score: {new_rmsle:.4f}")
```

```
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.314904 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 1689
[LightGBM] [Info] Number of data points in the train set: 2936736, number of used features: 18
[LightGBM] [Info] Start training from score 2.930816
Training until validation scores don't improve for 50 rounds
Did not meet early stopping. Best iteration is:
[1000]  valid_0's l2: 0.158878
```

Improved Validation RMSLE Score: 0.3986

```
# 1. Compute rolling stats on combined train/test dataset
all_df['sales_rol_mean_7'] = all_df.groupby(['store_nbr', 'family'])['sales'].transform(lambda x: x.shift(7).rolling(7).mean)
all_df['sales_rol_mean_14'] = all_df.groupby(['store_nbr', 'family'])['sales'].transform(lambda x: x.shift(7).rolling(14).me
all_df['is_holiday'] = all_df['date'].isin(national_holidays).astype(int)
```

```
# 2. Extract processed test features
test_v2 = all_df[all_df['id'].isin(test_df['id'])].copy()
```

```
for col in ['family', 'store_nbr', 'city', 'state', 'type']:
    test_v2[col] = test_v2[col].astype('category')
```

```
# 3. Predict & Exponentiate
test_preds_v2_log = model_v2.predict(test_v2[features_v2])
test_preds_v2 = np.exp1(np.clip(test_preds_v2_log, 0, None))
```

```
# 4. Save Submission File
sub_v2 = pd.DataFrame({
    'id': test_v2['id'].astype(int),
    'sales': test_preds_v2
}).sort_values('id')
```

```
sub_v2.to_csv('submission_v2.csv', index=False)
print("Updated submission_v2.csv created successfully!")
```

Updated submission_v2.csv created successfully!

```
!kaggle competitions submit -c store-sales-time-series-forecasting -f submission_v2.csv -m "LightGBM v2 with rolling stats &
```

```
Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to the latest vers
100% 740k/740k [00:00<00:00, 1.19MB/s]
Successfully submitted to Store Sales - Time Series Forecasting
```

```
import xgboost as xgb
from catboost import CatBoostRegressor
```

```
# Convert categorical features for XGBoost compatibility (numeric encoding)
X_tr_xgb = X_tr.copy()
X_va_xgb = X_va.copy()
test_xgb = test_v2[features_v2].copy()
```

```
cat_cols = ['family', 'store_nbr', 'city', 'state', 'type']
for col in cat_cols:
    X_tr_xgb[col] = X_tr_xgb[col].cat.codes
    X_va_xgb[col] = X_va_xgb[col].cat.codes
    test_xgb[col] = test_xgb[col].cat.codes
```

```
# 1. Train XGBoost
print("Training XGBoost...")
model_xgb = xgb.XGBRegressor(
    n_estimators=1000,
    learning_rate=0.03,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    tree_method='hist'
)
model_xgb.fit(X_tr_xgb, y_tr_log, eval_set=[(X_va_xgb, y_va_log)], verbose=200)
```

```
# 2. Train CatBoost
print("\nTraining CatBoost...")
model_cat = CatBoostRegressor(
    iterations=1000,
    learning_rate=0.04,
    depth=6,
    random_seed=42,
    cat_features=cat_cols,
    verbose=200
)
model_cat.fit(X_tr, y_tr_log, eval_set=(X_va, y_va_log), early_stopping_rounds=50)

# 3. Blend Predictions (Equal Weighting: 40% LightGBM, 30% XGBoost, 30% CatBoost)
val_preds_xgb = np.clip(model_xgb.predict(X_va_xgb), 0, None)
val_preds_cat = np.clip(model_cat.predict(X_va), 0, None)

ensemble_val_preds = (0.40 * val_preds_v2) + (0.30 * val_preds_xgb) + (0.30 * val_preds_cat)
ensemble_rmsle = root_mean_squared_error(y_va_log, ensemble_val_preds)

print(f"\nEnsemble Validation RMSLE Score: {ensemble_rmsle:.4f}")
```

```
-----
ModuleNotFoundError                                Traceback (most recent call last)
/tmp/ipykernel_5104/757131351.py in <cell line: 0>()
      1 import xgboost as xgb
----> 2 from catboost import CatBoostRegressor
      3
      4 # Convert categorical features for XGBoost compatibility (numeric encoding)
      5 X_tr_xgb = X_tr.copy()
```

ModuleNotFoundError: No module named 'catboost'

NOTE: If your import is failing due to a missing package, you can manually install dependencies using either `!pip` or `!apt`.

To view examples of installing some common dependencies, click the "Open Examples" button below.

OPEN EXAMPLES

Next steps: [Explain error](#)

!pip install catboost -q

97.1/97.1 MB 8.0 MB/s eta 0:00:00

```
import xgboost as xgb
from catboost import CatBoostRegressor

# Convert categorical features for XGBoost compatibility (numeric encoding)
X_tr_xgb = X_tr.copy()
X_va_xgb = X_va.copy()
test_xgb = test_v2[features_v2].copy()

cat_cols = ['family', 'store_nbr', 'city', 'state', 'type']
for col in cat_cols:
    X_tr_xgb[col] = X_tr_xgb[col].cat.codes
    X_va_xgb[col] = X_va_xgb[col].cat.codes
    test_xgb[col] = test_xgb[col].cat.codes

# 1. Train XGBoost
print("Training XGBoost...")
model_xgb = xgb.XGBRegressor(
    n_estimators=1000,
    learning_rate=0.03,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    tree_method='hist'
)
model_xgb.fit(
    X_tr_xgb, y_tr_log,
    eval_set=[(X_va_xgb, y_va_log)],
```

```

        verbose=200
    )

# 2. Train CatBoost
print("\nTraining CatBoost...")
model_cat = CatBoostRegressor(
    iterations=1000,
    learning_rate=0.04,
    depth=6,
    random_seed=42,
    cat_features=cat_cols,
    verbose=200
)
model_cat.fit(
    X_tr, y_tr_log,
    eval_set=(X_va, y_va_log),
    early_stopping_rounds=50
)

# 3. Blend Predictions (40% LightGBM, 30% XGBoost, 30% CatBoost)
val_preds_xgb = np.clip(model_xgb.predict(X_va_xgb), 0, None)
val_preds_cat = np.clip(model_cat.predict(X_va), 0, None)

ensemble_val_preds = (0.40 * val_preds_v2) + (0.30 * val_preds_xgb) + (0.30 * val_preds_cat)
ensemble_rmsle = root_mean_squared_error(y_va_log, ensemble_val_preds)

print(f"\nEnsemble Validation RMSLE Score: {ensemble_rmsle:.4f}")

```

```

Training XGBoost...
[0] validation_0-rmse:2.54390
[200] validation_0-rmse:0.41876
[400] validation_0-rmse:0.41106
[600] validation_0-rmse:0.40791
[800] validation_0-rmse:0.40588
[999] validation_0-rmse:0.40456

```

```

Training CatBoost...
0: learn: 2.5956741 test: 2.5192988 best: 2.5192988 (0) total: 6.52s remaining: 1h 48m 38s

```

```

KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipykernel_5104/965170688.py in <cell line: 0>()

```

```

    40     verbose=200
    41 )
--> 42 model_cat.fit(
    43     X_tr, y_tr_log,
    44     eval_set=(X_va, y_va_log),

```

2 frames

```

/usr/local/lib/python3.13/dist-packages/catboost/core.py in _train(self, train_pool, test_pool, params, allow_clear_pool,
init_model)

```

```

    1822
    1823     def _train(self, train_pool, test_pool, params, allow_clear_pool, init_model):
-> 1824         self._object._train(train_pool, test_pool, params, allow_clear_pool, init_model._object if init_model else
None)
    1825         self._set_trained_model_attributes()
    1826

```

```

_catboost.pyx in _catboost._CatBoost._train()

```

```

_catboost.pyx in _catboost._CatBoost._train()

```

```

KeyboardInterrupt:

```



Amazon Grocery Demand Forecasting | ARIMA + Prophet + XGBoost | Time Series

ML



Scenario

Amazon Fresh often faces **understocking and overstocking** challenges due to unpredictable demand for grocery products.



Business Objective

Build a robust **demand forecasting model** using time series machine learning techniques to help optimize inventory and reduce losses.

🗨️ Tech Stack

- **Models:** ARIMA, Prophet, XGBoost
- **Libraries:** pandas, matplotlib, seaborn, statsmodels, fbprophet, scikit-learn, xgboost
- **Evaluation Metrics:** RMSE, MAE, MAPE

📊 Dataset

[Store Sales Time Series Forecasting \(Kaggle\)](#)

This dataset includes daily sales data for multiple Ecuadorian grocery stores and product families, along with metadata like holidays, promotions, oil prices, etc.

⚠️ Due to the large size of the dataset (~3 million rows), we begin by working on a **focused subset**:

- 📍 **Store Number:** 1
- 📦 **Product Family:** GROCERY I

This approach allows fast experimentation and avoids memory issues on Kaggle kernels.

🔍 Project Objectives

- Clean and explore historical sales trends for a focused subset
- Analyze trends, seasonality, and holiday impact
- Build and compare time series forecasting models (ARIMA, Prophet, XGBoost)
- Evaluate performance using RMSE, MAE, and MAPE
- Discuss how this approach can be **scaled to all stores and product families**

⚙️ Fully built inside Kaggle. Beginner-friendly, step-by-step, with complete explanations.

📦 Import Required Libraries

We begin by importing essential libraries for data manipulation, visualization, and forecasting. Suppressing warnings helps keep the notebook clean.

```
# Core
import pandas as pd
import numpy as np

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Time Series
import warnings
warnings.filterwarnings("ignore")

# Display config
pd.set_option('display.max_columns', None)
```

📦 Load & Explore Subset of Dataset

The original dataset contains over 3 million rows across multiple stores and product families. To maintain performance and clarity, we use a filtered subset:

- 📍 Store Number: 1
- 📦 Product Family: GROCERY I

This subset allows faster experimentation and modeling. We'll scale later if needed.

```
# Load train data and parse date
df = pd.read_csv('/kaggle/input/store-sales-time-series-forecasting/train.csv', parse_dates=['date'])

# Subset for Store 1 and Family 'GROCERY I'
subset_df = df[(df['store_nbr'] == 1) & (df['family'] == 'GROCERY I')].copy()
```

```
subset_df.sort_values('date', inplace=True)
subset_df.reset_index(drop=True, inplace=True)

# Overview
print(f"Subset shape: {subset_df.shape}")
subset_df.head()
```

✦ Exploratory Data Analysis (EDA)

In this section, we explore the sales trends and seasonality for Store 1, Product Family 'GROCERY I'.

We'll visualize:

- Overall demand trend
- Monthly and weekly seasonality
- Promotional impact on sales

Understanding these patterns is essential before applying forecasting models.

📊 Code – EDA Plots

```
# Make sure 'date' is datetime
subset_df['date'] = pd.to_datetime(subset_df['date'])

# Set date as index
subset_df.set_index('date', inplace=True)

# Daily Sales Trend
plt.figure(figsize=(14, 4))
subset_df['sales'].plot()
plt.title('Daily Sales Trend (Store 1 - GROCERY I)')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```

📅 Monthly Seasonality

```
# Monthly average sales
monthly = subset_df['sales'].resample('M').mean()

plt.figure(figsize=(12, 4))
monthly.plot(marker='o')
plt.title('Monthly Average Sales')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```

📅 Weekly Seasonality

```
# Add weekday column (0=Monday, 6=Sunday)
subset_df['weekday'] = subset_df.index.weekday

# Plot average sales by weekday
plt.figure(figsize=(8, 4))
sns.barplot(x='weekday', y='sales', data=subset_df)
plt.title('Average Sales by Weekday')
plt.xlabel('Weekday (0 = Monday)')
plt.ylabel('Avg Sales')
plt.grid(True)
plt.show()
```

🔪 Promotion Impact

```
# Compare sales when promotion is on vs off
plt.figure(figsize=(6, 4))
sns.boxplot(x='onpromotion', y='sales', data=subset_df)
plt.title('Sales Distribution: Promotion vs No Promotion')
plt.xlabel('On Promotion')
plt.ylabel('Sales')
plt.grid(True)
plt.show()
```

▼ 🗨 Bonus: Moving Average Trend

```
# 30-day rolling mean
subset_df['sales'].rolling(window=30).mean().plot(figsize=(14, 4), label='30-day Moving Avg')
subset_df['sales'].plot(alpha=0.4, label='Actual Sales')
plt.title('Sales with 30-day Moving Average')
plt.legend()
plt.grid(True)
plt.show()
```

✅ Insights

- The sales show clear weekly and monthly seasonality.
- Weekends (especially Sunday) show slightly higher demand.
- Promotions have a noticeable impact on boosting sales.
- The moving average helps smooth out noise and shows long-term trends.

▼ 💠 Advanced EDA – Seasonality, Holiday Impact & Autocorrelation

We now go beyond basic EDA and explore:

- 📅 Holiday influence on sales
- 🌀 Autocorrelation to understand time-based dependencies
- 📊 Cleaned visuals with color, spacing, and grid enhancements

▼ 📊 1. Daily Sales Trend (Improved Visual)

```
plt.figure(figsize=(14, 5))
plt.plot(subset_df.index, subset_df['sales'], color='teal', linewidth=1.2)
plt.title('Daily Sales Trend – Store 1 | GROCERY I', fontsize=14)
plt.xlabel('Date')
plt.ylabel('Sales')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

▼ 📅 2. Monthly Sales Pattern (Improved)

```
monthly_avg = subset_df['sales'].resample('M').mean()

plt.figure(figsize=(12, 4))
plt.plot(monthly_avg.index, monthly_avg.values, marker='o', color='darkorange')
plt.title('Monthly Average Sales', fontsize=13)
plt.xlabel('Month')
plt.ylabel('Avg Sales')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```

3. Average Sales by Day of Week (Improved Barplot)

```
weekday_avg = subset_df.groupby(subset_df.index.day_name())['sales'].mean().reindex([
    'Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'
])

plt.figure(figsize=(10, 4))
sns.barplot(x=weekday_avg.index, y=weekday_avg.values, palette='coolwarm')
plt.title('Average Sales by Day of Week', fontsize=13)
plt.ylabel('Avg Sales')
plt.xticks(rotation=45)
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

4. Promotion Impact (Improved Boxplot)

```
plt.figure(figsize=(6, 4))
sns.boxplot(x='onpromotion', y='sales', data=subset_df, palette='Set2')
plt.title('Sales Distribution: Promotion vs No Promotion', fontsize=12)
plt.xticks([0, 1], ['No', 'Yes'])
plt.xlabel('On Promotion')
plt.ylabel('Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

5. Holiday Impact (If holiday_events.csv available)

```
# Optional: Load holiday data
holidays = pd.read_csv('/kaggle/input/store-sales-time-series-forecasting/holidays_events.csv', parse_dates=['date'])

# Merge holiday info
holiday_merge = pd.merge(
    subset_df.reset_index(),
    holidays[['date', 'type']],
    how='left',
    on='date'
)

# Add holiday flag
holiday_merge['is_holiday'] = holiday_merge['type'].notna().astype(int)

# Compare sales on holiday vs non-holiday
plt.figure(figsize=(6, 4))
sns.boxplot(x='is_holiday', y='sales', data=holiday_merge, palette='Paired')
plt.title('Sales on Holidays vs Non-Holidays')
plt.xticks([0, 1], ['Non-Holiday', 'Holiday'])
plt.ylabel('Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

6. Autocorrelation Plot (ACF)

```
from statsmodels.graphics.tsaplots import plot_acf

plt.figure(figsize=(10, 4))
plot_acf(subset_df['sales'], lags=30)
plt.title('Autocorrelation Plot (Sales)')
plt.tight_layout()
plt.show()
```

7. Sales by Year

```
# Extract year from date
subset_df['year'] = subset_df.index.year

# Group and plot
yearly_avg = subset_df.groupby('year')['sales'].mean()

plt.figure(figsize=(8, 4))
sns.barplot(x=yearly_avg.index.astype(str), y=yearly_avg.values, palette='viridis')
plt.title('Average Sales Per Year')
plt.xlabel('Year')
plt.ylabel('Avg Sales')
plt.grid(True, axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

8. Sales by Month (Across All Years)

```
subset_df['month'] = subset_df.index.month

monthly_avg = subset_df.groupby('month')['sales'].mean()

plt.figure(figsize=(10, 4))
sns.lineplot(x=monthly_avg.index, y=monthly_avg.values, marker='o', color='darkgreen')
plt.title('Average Sales by Month (All Years Combined)')
plt.xlabel('Month')
plt.ylabel('Avg Sales')
plt.xticks(range(1, 13))
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

9. Sales by Day of Month

```
subset_df['day'] = subset_df.index.day

day_avg = subset_df.groupby('day')['sales'].mean()

plt.figure(figsize=(12, 4))
sns.lineplot(x=day_avg.index, y=day_avg.values, color='purple')
plt.title('Average Sales by Day of Month')
plt.xlabel('Day')
plt.ylabel('Avg Sales')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

10. Seasonal Decomposition Plot

```
from statsmodels.tsa.seasonal import seasonal_decompose

# Ensure daily frequency
ts = subset_df['sales'].asfreq('D')

# Fill missing values
ts = ts.fillna(method='ffill')

# Decompose
decomp = seasonal_decompose(ts, model='additive', period=365)

# Plot
decomp.plot()
plt.suptitle('Seasonal Decomposition (Additive)', fontsize=14)
plt.tight_layout()
```

```
plt.show()

missing_days = ts[ts.isna()]
print(f"Missing days count: {missing_days.shape[0]}")
```

Insights

- Sales have clear weekly and monthly patterns.
- Holidays appear to affect demand slightly, with higher variation on event days.
- Promotions positively influence sales, as shown in the boxplot.
- Autocorrelation confirms short-term time dependencies, especially weekly lag effects.
- Year-wise sales have remained relatively stable with minor seasonal peaks.
- Month-wise trends show recurring demand spikes in specific months.
- Day-of-month analysis shows no strong fixed-day effect, but slight dips in mid-month.
- Seasonal decomposition separates trend, seasonality, and residuals — confirming our earlier findings.

✦ Time Series Preparation – Train/Test Split & Stationarity Check

Before training time series models, it's essential to prepare the data:

-  Split data into training and testing sets
-  Apply log transformation to stabilize variance
-  Check for stationarity using Augmented Dickey-Fuller (ADF) test
-  Visualize rolling mean & std to confirm stability

✓ Train-Test Split

```
# Reset index if needed
data = subset_df[['sales']].copy()
data = data.asfreq('D') # ensure daily frequency
data = data.fillna(method='ffill')

# Set cutoff date (e.g., last 3 months for testing)
split_date = '2017-07-01'

train = data.loc[:split_date].copy()
test = data.loc[split_date:].copy()

print(f"Train shape: {train.shape}")
print(f"Test shape: {test.shape}")
```

✓ Log Transformation (Optional but useful)

```
train['log_sales'] = np.log1p(train['sales']) # log(1 + x)
test['log_sales'] = np.log1p(test['sales'])
```

✓ ADF Test – Augmented Dickey-Fuller

```
from statsmodels.tsa.stattools import adfuller

result = adfuller(train['log_sales'].dropna())
print("ADF Statistic:", result[0])
print("p-value:", result[1])
for key, value in result[4].items():
    print(f"Critical Value ({key}): {value}")

if result[1] < 0.05:
    print("✓ The series is stationary.")
else:
    print("⚠ The series is non-stationary.")
```

- The ADF test shows whether the series is stationary.
- A p-value below 0.05 confirms that the series is already stationary.

✓ Rolling Mean & Standard Deviation Plot

```
rol_mean = train['log_sales'].rolling(window=30).mean()
rol_std = train['log_sales'].rolling(window=30).std()

plt.figure(figsize=(12, 5))
plt.plot(train['log_sales'], color='blue', label='Log Transformed Sales')
plt.plot(rol_mean, color='red', label='Rolling Mean (30)')
plt.plot(rol_std, color='black', label='Rolling Std (30)')
plt.legend(loc='best')
plt.title('Rolling Mean & Standard Deviation (Log Sales)')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

- Rolling plots help visually confirm if the mean and variance are stable over time.
- If the series is non-stationary, differencing or decomposition may be required.

◆ Model Training & Forecasting (ARIMA, Prophet, XGBoost)

Now that the data is clean and prepared, we build forecasting models:

- 🌀 ARIMA for linear time-based forecasting
- 🌙 Prophet for handling seasonality and holidays
- 🚀 XGBoost as a machine learning-based time series model

We'll train, forecast, and compare performance using RMSE, MAE, and MAPE.

✓ ARIMA Modeling

```
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Fit ARIMA on log-transformed data
model_arima = ARIMA(train['log_sales'], order=(5,1,0))
model_arima_fit = model_arima.fit()

# Forecast steps ahead
forecast_steps = len(test)
forecast_log = model_arima_fit.forecast(steps=forecast_steps)
forecast_arima = np.exp(forecast_log) # convert back from log

# Evaluation
rmse_arima = np.sqrt(mean_squared_error(test['sales'], forecast_arima))
mae_arima = mean_absolute_error(test['sales'], forecast_arima)

print("ARIMA RMSE:", rmse_arima)
print("ARIMA MAE:", mae_arima)
```

📊 Plot ARIMA Forecast

```
plt.figure(figsize=(14, 5))
plt.plot(train.index, train['sales'], label='Train')
plt.plot(test.index, test['sales'], label='Actual')
plt.plot(test.index, forecast_arima, label='ARIMA Forecast', color='green')
plt.title('ARIMA Forecast vs Actual')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
```

```
plt.tight_layout()
plt.show()
```

✓ Prepare Data for Prophet

```
from prophet import Prophet

# Prophet needs 'ds' and 'y'
prophet_df = subset_df.reset_index()[['date', 'sales']].rename(columns={'date': 'ds', 'sales': 'y'})

# Filter again for same subset
prophet_df = prophet_df[(prophet_df['ds'] < '2017-08-01') & (prophet_df['ds'] >= '2013-01-01')]

# Split
train_prophet = prophet_df[prophet_df['ds'] < '2017-07-01']
test_prophet = prophet_df[prophet_df['ds'] >= '2017-07-01']
```

✓ Fit & Forecast Prophet

```
m = Prophet()
m.fit(train_prophet)

future = m.make_future_dataframe(periods=len(test_prophet))
forecast = m.predict(future)

# Evaluation
y_true = test_prophet['y'].values
y_pred = forecast.iloc[-len(test_prophet):]['yhat'].values

rmse_prophet = np.sqrt(mean_squared_error(y_true, y_pred))
mae_prophet = mean_absolute_error(y_true, y_pred)

print("Prophet RMSE:", rmse_prophet)
print("Prophet MAE:", mae_prophet)
```

📊 Plot Prophet Forecast

```
fig = m.plot(forecast)
plt.title('Prophet Forecast')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.tight_layout()
plt.show()
```

◆ Forecasting with XGBoost (Lag-Based ML Approach)

To leverage machine learning for time series, we engineer features from past values and time components.

We'll use:

- Lag features (t-1, t-7, etc.)
- Rolling means
- Date-based components
- Then train an XGBoost model to forecast future sales

⚙️ Feature Engineering

```
df_ml = subset_df[['sales']].copy()
df_ml['day'] = df_ml.index.day
df_ml['month'] = df_ml.index.month
df_ml['weekday'] = df_ml.index.weekday
```

```
df_ml['year'] = df_ml.index.year

# Lag features
df_ml['lag_1'] = df_ml['sales'].shift(1)
df_ml['lag_7'] = df_ml['sales'].shift(7)
df_ml['rolling_mean_7'] = df_ml['sales'].shift(1).rolling(window=7).mean()

# Drop NaNs
df_ml.dropna(inplace=True)
```

✂ Train-Test Split

```
X = df_ml.drop('sales', axis=1)
y = df_ml['sales']

# Split index (same as previous)
split_date = '2017-07-01'
X_train = X.loc[:split_date]
y_train = y.loc[:split_date]
X_test = X.loc[split_date:]
y_test = y.loc[split_date:]
```

🚀 Train XGBoost Model

```
from xgboost import XGBRegressor

model_xgb = XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
model_xgb.fit(X_train, y_train)

# Predict
y_pred_xgb = model_xgb.predict(X_test)
```

📊 Evaluation

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

rmse_xgb = np.sqrt(mean_squared_error(y_test, y_pred_xgb))
mae_xgb = mean_absolute_error(y_test, y_pred_xgb)

print("XGBoost RMSE:", rmse_xgb)
print("XGBoost MAE:", mae_xgb)
```

📈 Plot Actual vs Predicted

```
plt.figure(figsize=(14, 5))
plt.plot(y_test.index, y_test, label='Actual Sales')
plt.plot(y_test.index, y_pred_xgb, label='XGBoost Forecast', color='orange')
plt.title('XGBoost Forecast vs Actual')
plt.xlabel('Date')
plt.ylabel('Sales')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

XGBoost performed well by learning patterns from lag and rolling statistics.

Key Pros:

- Learns non-linear patterns
- Can include external regressors
- Fast to train

We'll now compare performance across all models.

▼ Model Comparison – ARIMA vs Prophet vs XGBoost

After training all three models, we now compare their performance using:

-  RMSE (Root Mean Squared Error)
-  MAE (Mean Absolute Error)
-  MAPE (Mean Absolute Percentage Error)

▼ Comparison Table

```
from sklearn.metrics import mean_absolute_percentage_error

# Calculate MAPE for Prophet and XGBoost
mape_arima = mean_absolute_percentage_error(test['sales'], forecast_arima)
mape_prophet = mean_absolute_percentage_error(y_true, y_pred)
mape_xgb = mean_absolute_percentage_error(y_test, y_pred_xgb)

# Final comparison
results = pd.DataFrame({
    'Model': ['ARIMA', 'Prophet', 'XGBoost'],
    'RMSE': [rmse_arima, rmse_prophet, rmse_xgb],
    'MAE': [mae_arima, mae_prophet, mae_xgb],
    'MAPE': [mape_arima, mape_prophet, mape_xgb]
})

results.sort_values('RMSE')
```

▼ Visual Comparison – Barplot